

```
#importing app from the __init__.py file, path can be obtained
from the above statement
from article import app
manager = Manager(app)

# main page can load either url 127.0.0.0:5000/ or
127.0.0.0:5000/index
@app.route('/')
@app.route('/index')
def index():
    return render_template("index.html", title = "Myapp home")

#About page
@app.route('/about')
def about():
    return render_template("about.html", title= "About")

# load posts page and displays the content from the post array
@app.route('/posts')
def posts():
    posts = [
        {
            "title" : "Hello World",
            "body" : "Welcome to python flask tutorial"
        },
        {
            "title" : "Sample post",
            "body" : "Sample post for testing python flask "
        }
    ]
    return render_template("posts.html", title = "Posts",
posts = posts)

# Create a new command 'runserver' using Manager and set host
and enable debugger module.
# Debugger module only for developing not for production
server
manager.add_command('runserver', Server(
    use_debugger = True,
    use_reloader = True,
    host = '127.0.0.1'
))
#manager.run will start the server, which holds the app object
also from the file __init__.py registered using 'Manager'
if __name__ == "__main__":
    manager.run()
```

Import all necessary modules for our app. The *render_template* module in Flask uses the Jinja2 template, which automatically locates the file from the *templates* directory in the app. You can pass the template name and arguments as well. *url_for* is good for building URLs. Instead of hard code, the URLs passing the function names will take care of everything and arguments will also be accepted.

The application path can be obtained from modules *os* and

sys, and *app* object is imported from the *__init__.py* file.

manage.py will take care of routing and content in our application. Routing can be done using the *@app.route()* method, which takes string arguments and will be appended to the URL, e.g., *@app.route('/index')*. You can load the index page with the help of *http://127.0.0.0:5000/index*.

render_template will, by default, look for HTML pages in the template directory, which takes the first argument as a page name and passing key; value pairs are also possible in the *render_template* method and values can be received using its key.

Manager module in Flask-script allows the user to create a new command. Using the *add_command* we can set up the server and new command. Here, we used *runserver* as our new command, and server settings like debugging, reloading and application IP address. Enabling debugger (*use_debugger = True*) is recommended for the purpose of testing and not for the production environment. It will display an error message if something goes wrong, and reloader (*use_reloader = True*) will automatically reload the applications after we have made changes to the code.

You can run the application using */manage.py runserver*. *runserver* is our own command, which has been created using the *Manager* module.

base.html

```
<!DOCTYPE html>
<html>
  <head>
    {% if title %}
      <title> {{title}}</title>
    {% else %}
      <title> Myapp home</title>
    {% endif %}
    <link rel="stylesheet" type="text/css" href="{{ url_
for('static', filename='css/style.css') }}">
  </head>
  <body>
    <header>
      <h1><a href="{{url_for('index')}}">MyApp</a></h1>
    </header>
    <div class="menu">
      <ul>
        <li><a href="{{url_for('index')}}">Home</a></li>
        <li><a href="{{url_for('about')}}">About</a></li>
        <li><a href="{{url_for('posts')}}">Post</a></li>
      </ul>
    </div>
    <div class="container">
      {% block content %}{% endblock %}
    </div>
  </body>
```